

LIN7076 – Foundations of Computational Linguistics

Naive Bayes

Adèle Hénot-Mortier

05/10/2026

Queen Mary University of London

Binary classification: some applications



- Positive? Negative?
- Neutral? Hateful?
- Neutral? Sarcastic?
- Real news? Fake news?
- Unbiased? Biased?

Binary classification: the task

- Given a piece of text, predict if it belongs to class 0 (e.g. **neutral**) or to class 1 (e.g. **hateful**). This is **binary classification**.
- **Supervised learning** *via* a set of (text, class) training examples.
- A few techniques used for binary classification: Naive Bayes, logistic regression, support vector machines, and of course, neural nets.¹
- We'll cover on **Naive Bayes** (today) and **logistic regression** (in 2 weeks).
- We will focus on binary classification, but in fact **Naive Bayes extends to more than 2 classes!**

¹There's also k -nearest neighbors, but it's unsupervised.

Binary classification: the task

- Given a piece of text, predict if it belongs to class 0 (e.g. **neutral**) or to class 1 (e.g. **hateful**). This is **binary classification**.
- **Supervised learning** *via* a set of (text, class) training examples.
- A few techniques used for binary classification: Naive Bayes, logistic regression, support vector machines, and of course, neural nets.¹
- We'll cover on **Naive Bayes** (today) and **logistic regression** (in 2 weeks).
- We will focus on binary classification, but in fact **Naive Bayes extends to more than 2 classes!**

¹There's also k -nearest neighbors, but it's unsupervised.

Binary classification: the task

- Given a piece of text, predict if it belongs to class 0 (e.g. **neutral**) or to class 1 (e.g. **hateful**). This is **binary classification**.
- **Supervised learning** *via* a set of (text, class) training examples.
- A few techniques used for binary classification: Naive Bayes, logistic regression, support vector machines, and of course, neural nets.¹
- We'll cover on **Naive Bayes** (today) and **logistic regression** (in 2 weeks).
- We will focus on binary classification, but in fact **Naive Bayes** extends to more than 2 classes!

¹There's also k -nearest neighbors, but it's unsupervised.

Binary classification: the task

- Given a piece of text, predict if it belongs to class 0 (e.g. **neutral**) or to class 1 (e.g. **hateful**). This is **binary classification**.
- **Supervised learning** *via* a set of (text, class) training examples.
- A few techniques used for binary classification: Naive Bayes, logistic regression, support vector machines, and of course, neural nets.¹
- We'll cover on **Naive Bayes** (today) and **logistic regression** (in 2 weeks).
- We will focus on binary classification, but in fact **Naive Bayes extends to more than 2 classes!**

¹There's also k -nearest neighbors, but it's unsupervised.

Binary classification: the task

- Given a piece of text, predict if it belongs to class 0 (e.g. **neutral**) or to class 1 (e.g. **hateful**). This is **binary classification**.
- **Supervised learning** *via* a set of (text, class) training examples.
- A few techniques used for binary classification: Naive Bayes, logistic regression, support vector machines, and of course, neural nets.¹
- We'll cover on **Naive Bayes** (today) and **logistic regression** (in 2 weeks).
- We will focus on binary classification, but in fact **Naive Bayes extends to more than 2 classes!**

¹There's also k -nearest neighbors, but it's unsupervised.

Setting up the model

Naive Bayes: core intuition

- Different pieces of text have **different characteristics**, for instance,

Naive Bayes: core intuition

- Different pieces of text have **different characteristics**, for instance, they vary in length, capitalization, use of punctuation, and of course, in the words they use.

Naive Bayes: core intuition

- Different pieces of text have **different characteristics**, for instance, they vary in length, capitalization, use of punctuation, and of course, in the words they use.
- Some of these features seem very relevant to certain classification tasks; for instance, a text in all caps with lots of strong punctuation and offensive words, is probably hateful. A text without these features is more likely to be neutral.

Naive Bayes: core intuition

- Different pieces of text have **different characteristics**, for instance, they vary in length, capitalization, use of punctuation, and of course, in the words they use.
- Some of these features seem very relevant to certain classification tasks; for instance, a text in all caps with lots of strong punctuation and offensive words, is probably hateful. A text without these features is more likely to be neutral.
- To determine which class a text belongs to, one can then think of how likely it is to display the features it has, were it from class 0, or from class 1.

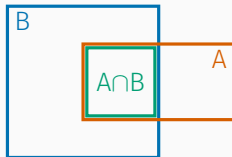
A piece of text belongs to the class that makes the features it displays more likely!

Conditional probabilities, again!

- The **conditional probability** of an event A , given an event B , can be computed using the joint probability of A and B , and B 's probability.
- Symmetrically, the conditional probability of B , given A , can be computed using the same joint probability, and A 's probability.

$$P(A|B) = \frac{P(A, B)}{P(B)} = \frac{P(A \cap B)}{P(B)}$$

$$P(B|A) = \frac{P(A, B)}{P(A)} = \frac{P(A \cap B)}{P(A)}$$

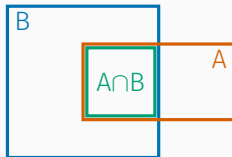


Conditional probabilities, again!

- The **conditional probability** of an event A , given an event B , can be computed using the joint probability of A and B , and B 's probability.
- Symmetrically, the conditional probability of B , given A , can be computed **using the same joint probability**, and A 's probability.

$$P(A|B) = \frac{P(A, B)}{P(B)} = \frac{P(A \cap B)}{P(B)}$$

$$P(B|A) = \frac{P(A, B)}{P(A)} = \frac{P(A \cap B)}{P(A)}$$



Classification as conditional probability optimization

- Suppose we have a piece of text (“document”) d , that we want to classify as neutral (c_0) or hateful (c_1).
- We want to figure out which class is the most likely to represent d , meaning, we want to compute $P(c_0|d)$ and $P(c_1|d)$, and assign d the class that is more likely.

$$\hat{c}(d) = \begin{cases} c_0 & \text{if } P(c_0|d) > P(c_1|d) \\ c_1 & \text{if } P(c_0|d) < P(c_1|d) \end{cases}$$

- We can “compress” and generalize this with the **argmax** notation: $\operatorname{argmax}_x f(x)$ returns the element x that maximizes the function f ; here, the class c that maximizes the function $P(c|d)$ for a fixed d .

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

Classification as conditional probability optimization

- Suppose we have a piece of text (“document”) d , that we want to classify as neutral (c_0) or hateful (c_1).
- We want to figure out **which class is the most likely to represent d** , meaning, we want to compute $P(c_0|d)$ and $P(c_1|d)$, and assign d the class that is more likely.

$$\hat{c}(d) = \begin{cases} c_0 & \text{if } P(c_0|d) > P(c_1|d) \\ c_1 & \text{if } P(c_0|d) < P(c_1|d) \end{cases}$$

- We can “compress” and generalize this with the **argmax** notation: $\operatorname{argmax}_x f(x)$ returns the element x that maximizes the function f ; here, the class c that maximizes the function $P(c|d)$ for a fixed d .

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

Classification as conditional probability optimization

- Suppose we have a piece of text (“document”) d , that we want to classify as neutral (c_0) or hateful (c_1).
- We want to figure out **which class is the most likely to represent** d , meaning, we want to compute $P(c_0|d)$ and $P(c_1|d)$, and assign d the class that is more likely.

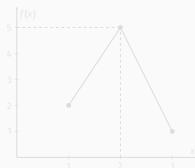
$$\hat{c}(d) = \begin{cases} c_0 & \text{if } P(c_0|d) > P(c_1|d) \\ c_1 & \text{if } P(c_0|d) < P(c_1|d) \end{cases}$$

- We can “compress” and generalize this with the **argmax** notation: $\operatorname{argmax}_x f(x)$ returns the element x that maximizes the function f ; here, the class c that maximizes the function $P(c|d)$ for a fixed d .

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

Argmax clarification

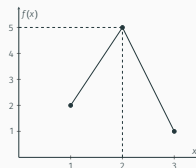
- To better understand the argmax, let's go back to the max operator. $\max_{x \in D} f(x)$ returns the value(s) of $f(x)$ that are maximal within the domain D ; the maxima.
- The plot below for instance, shows that the max of f is 5 for $x = 2$, within the domain $D = \{1, 2, 3\}$.



- Now, $\operatorname{argmax}_{x \in D} f(x)$ returns the **maximizer(s)** of f : the values of x within the domain D s.t. $f(x)$ is maximal.
- The plot above for instance, shows that the argmax of f is 2, because for $x = 2$, $f(x)$ is maximal. To find the argmax, find the maxima, and **return the values achieving these maxima.**

Argmax clarification

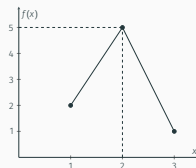
- To better understand the argmax, let's go back to the max operator. $\max_{x \in D} f(x)$ returns the value(s) of $f(x)$ that are maximal within the domain D ; the maxima.
- The plot below for instance, shows that the max of f is 5 for $x = 2$, within the domain $D = \{1, 2, 3\}$.



- Now, $\operatorname{argmax}_{x \in D} f(x)$ returns the **maximizer(s)** of f : the values of x within the domain D s.t. $f(x)$ is maximal.
- The plot above for instance, shows that the argmax of f is 2, because for $x = 2$, $f(x)$ is maximal. To find the argmax, find the maxima, and return the values achieving these maxima.

Argmax clarification

- To better understand the argmax, let's go back to the max operator. $\max_{x \in D} f(x)$ returns the value(s) of $f(x)$ that are maximal within the domain D ; the maxima.
- The plot below for instance, shows that the max of f is 5 for $x = 2$, within the domain $D = \{1, 2, 3\}$.



- Now, $\operatorname{argmax}_{x \in D} f(x)$ returns the **maximizer(s)** of f : the values of x within the domain D s.t. $f(x)$ is maximal.
- The plot above for instance, shows that the argmax of f is 2, because for $x = 2$, $f(x)$ is maximal. To find the argmax, find the maxima, and **return the values achieving these maxima**.

Deriving the Bayes formula for document classification

- To compare $P(c_0|d)$ and $P(c_1|d)$, we need to **approximate** them!
- Here is the trick: **rewrite the joint** $P(c \cap d)$ using $P(d|c)$ (the probability of a document, *given a class*) and $P(c)$, using the conditional probability formula:

$$\begin{aligned}P(d|c) &= \frac{P(d \cap c)}{P(c)} \\P(d \cap c) &= P(d|c)P(c)\end{aligned}$$

- Then express the target probability $P(c|d)$ as a function of $P(d|c)$, $P(d)$, and $P(c)$!

$$P(c|d) = \frac{P(d \cap c)}{P(d)} = \frac{P(d|c)P(c)}{P(d)} \quad \text{Bayes' Theorem}$$

- $P(d|c)P(c)$ will prove **easier to approximate** than the joint $P(c \cap d)$.

Deriving the Bayes formula for document classification

- To compare $P(c_0|d)$ and $P(c_1|d)$, we need to **approximate** them!
- Here is the trick: **rewrite the joint** $P(c \cap d)$ using $P(d|c)$ (the probability of a document, *given a class*) and $P(c)$, using the conditional probability formula:

$$\begin{aligned}P(d|c) &= \frac{P(d \cap c)}{P(c)} \\P(d \cap c) &= P(d|c)P(c)\end{aligned}$$

- Then express the target probability $P(c|d)$ as a function of $P(d|c)$, $P(d)$, and $P(c)$!

$$P(c|d) = \frac{P(d \cap c)}{P(d)} = \frac{P(d|c)P(c)}{P(d)} \quad \text{Bayes' Theorem}$$

- $P(d|c)P(c)$ will prove **easier to approximate** than the joint $P(c \cap d)$.

Deriving the Bayes formula for document classification

- To compare $P(c_0|d)$ and $P(c_1|d)$, we need to **approximate** them!
- Here is the trick: **rewrite the joint** $P(c \cap d)$ using $P(d|c)$ (the probability of a document, *given a class*) and $P(c)$, using the conditional probability formula:

$$\begin{aligned}P(d|c) &= \frac{P(d \cap c)}{P(c)} \\P(d \cap c) &= P(d|c)P(c)\end{aligned}$$

- Then express the target probability $P(c|d)$ as a function of $P(d|c)$, $P(d)$, and $P(c)$!

$$P(c|d) = \frac{P(d \cap c)}{P(d)} = \frac{P(d|c)P(c)}{P(d)} \quad \text{Bayes' Theorem}$$

- $P(d|c)P(c)$ will prove **easier to approximate** than the joint $P(c \cap d)$.

Deriving the Bayes formula for document classification

- To compare $P(c_0|d)$ and $P(c_1|d)$, we need to **approximate** them!
- Here is the trick: **rewrite the joint** $P(c \cap d)$ using $P(d|c)$ (the probability of a document, *given a class*) and $P(c)$, using the conditional probability formula:

$$\begin{aligned}P(d|c) &= \frac{P(d \cap c)}{P(c)} \\P(d \cap c) &= P(d|c)P(c)\end{aligned}$$

- Then express the target probability $P(c|d)$ as a function of $P(d|c)$, $P(d)$, and $P(c)$!

$$P(c|d) = \frac{P(d \cap c)}{P(d)} = \frac{P(d|c)P(c)}{P(d)} \quad \text{Bayes' Theorem}$$

- $P(d|c)P(c)$ will prove **easier to approximate** than the joint $P(c \cap d)$.

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, we can conveniently ignore the denominators $P(d)$.

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior** $P(c|d)$), is proportional to:
 - how likely the document is, given that class (**likelihood** $P(d|c)$);
 - and to how probable the class is before observing the document (**prior** $P(c)$).

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, **we can conveniently ignore the denominators $P(d)$.**

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior** $P(c|d)$), is proportional to:
 - how likely the document is, given that class (**likelihood** $P(d|c)$);
 - and to how probable the class is before observing the document (**prior** $P(c)$).

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, **we can conveniently ignore the denominators $P(d)$.**

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior** $P(c|d)$), is proportional to:
 - how likely the document is, given that class (**likelihood** $P(d|c)$);
 - and to how probable the class is before observing the document (**prior** $P(c)$).

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, **we can conveniently ignore the denominators $P(d)$.**

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior $P(c|d)$**), is proportional to:
 - how likely the document is, given that class (**likelihood $P(d|c)$**);
 - and to how probable the class is before observing the document (**prior $P(c)$**).

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, **we can conveniently ignore the denominators $P(d)$.**

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior** $P(c|d)$), is proportional to:
 - how likely the document is, given that class (**likelihood** $P(d|c)$);
 - and to how probable the class is before observing the document (**prior** $P(c)$).

Analyzing Bayes' Theorem

$$P(c_0|d) = \frac{P(d|c_0)P(c_0)}{P(d)}$$

$$P(c_1|d) = \frac{P(d|c_1)P(c_1)}{P(d)}$$

- To compare the above conditional probabilities, **we can conveniently ignore the denominators $P(d)$.**

$$\underbrace{P(c|d)}_{\text{posterior}} \propto \overbrace{P(d|c)}^{\text{likelihood}} \underbrace{P(c)}_{\text{prior}}$$

- The above equation says that the probability of a class, after observing a document (**posterior** $P(c|d)$), is proportional to:
 - how likely the document is, given that class (**likelihood** $P(d|c)$);
 - and to how probable the class is before observing the document (**prior** $P(c)$).

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.
- Suppose we have a document d , whose features make it **very likely to be harmful**, say $P(d|c_1) = .99$. Still it's not completely impossible d is neutral, say $P(d|c_0) = .05$.

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.
- Suppose we have a document d , whose features make it **very likely to be harmful**, say $P(d|c_1) = .99$. Still it's not completely impossible d is neutral, say $P(d|c_0) = .05$.
- Which class should we assign d ?

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.
- Suppose we have a document d , whose features make it **very likely to be harmful**, say $P(d|c_1) = .99$. Still it's not completely impossible d is neutral, say $P(d|c_0) = .05$.
- Which class should we assign d ?

$$P(c_0|d) \propto .05 * .9999 \simeq .05$$

$$P(c_1|d) \propto .99 * .0001 \simeq .0001$$

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.
- Suppose we have a document d , whose features make it **very likely to be harmful**, say $P(d|c_1) = .99$. Still it's not completely impossible d is neutral, say $P(d|c_0) = .05$.
- Which class should we assign d ?

$$P(c_0|d) \propto .05 * .9999 \simeq .05$$

$$P(c_1|d) \propto .99 * .0001 \simeq .0001$$

- Counterintuitively, we should predict d to be **neutral** according to Naive Bayes!

Illustrating the interplay between prior and likelihood

- Suppose harmful (c_1) documents represent .01% of all documents. $P(c_1) = .0001$, $P(c_0) = .9999$.
- Suppose we have a document d , whose features make it **very likely to be harmful**, say $P(d|c_1) = .99$. Still it's not completely impossible d is neutral, say $P(d|c_0) = .05$.
- Which class should we assign d ?

$$P(c_0|d) \propto .05 * .9999 \simeq .05$$

$$P(c_1|d) \propto .99 * .0001 \simeq .0001$$

- Counterintuitively, we should predict d to be **neutral** according to Naive Bayes!
- This illustrates how priors influence “rational” decision making, sometimes against what likelihood suggests. **Humans are notoriously bad at taking priors into account!**

Introducing features

$$P(c|d) \propto P(d|c)P(c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

$$= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(d|c)P(c)$$

- Our predicted class is the one for which d is the most likely, weighted by how probable the class is independently of d .
- How to measure how likely a document is given a class? Documents with certain **features**, tend to belong to certain classes...

$P(d|c) = P(F(d)|c)$ with $F(d) = (f_1, \dots, f_k)$ d 's relevant features

- For instance, harmful documents contain harmful words... so the f_i can be words. This may sound idle, but dealing with individual words will help us further break down $P(d|c)$.

Introducing features

$$P(c|d) \propto P(d|c)P(c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

$$= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(d|c)P(c)$$

- Our predicted class is the one for which d is the most likely, weighted by how probable the class is independently of d .
- How to measure how likely a document is given a class?
Documents with certain **features**, tend to belong to certain classes...

$P(d|c) = P(F(d)|c)$ with $F(d) = (f_1, \dots, f_k)$ d 's relevant features

- For instance, harmful documents contain harmful words... so the f_i can be words. This may sound idle, but dealing with individual words will help us further break down $P(d|c)$.

Introducing features

$$P(c|d) \propto P(d|c)P(c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d)$$

$$= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(d|c)P(c)$$

- Our predicted class is the one for which d is the most likely, weighted by how probable the class is independently of d .
- How to measure how likely a document is given a class? Documents with certain **features**, tend to belong to certain **classes**...

$P(d|c) = P(F(d)|c)$ with $F(d) = (f_1, \dots, f_k)$ d 's relevant features

- For instance, harmful documents contain harmful words... so the f_i can be words. This may sound idle, but dealing with individual words will help us further break down $P(d|c)$.

Introducing features

$$\begin{aligned}P(c|d) &\propto P(d|c)P(c) \\ \hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(d|c)P(c)\end{aligned}$$

- Our predicted class is the one for which d is the most likely, weighted by how probable the class is independently of d .
- How to measure how likely a document is given a class? Documents with certain **features**, tend to belong to certain **classes**...

$P(d|c) = P(F(d)|c)$ with $F(d) = (f_1, \dots, f_k)$ d 's relevant features

- For instance, harmful documents contain harmful words... so the f_i can be words. This may sound idle, but dealing with individual words will help us further break down $P(d|c)$.

The Bag of Word assumption

- Documents are “featurized” as **the multiset of their words**. A “multiset” is unordered like a set, but keeps track of multiple occurrences.
- It’s like writing each individual word of your document on separate pieces of paper, and tossing them all into a bag. For any document d , we call $BoW(d)$ its corresponding **Bag of Words**.

$$P(d|c) \simeq P(BoW(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(BoW(d)|c)P(c)$$

- Can you think of a classification task and a specific document (=sentence) for which losing positional information would likely affect classification?

The Bag of Word assumption

- Documents are “featurized” as **the multiset of their words**. A “multiset” is unordered like a set, but keeps track of multiple occurrences.
- It’s like writing each individual word of your document on separate pieces of paper, and tossing them all into a bag. For any document d , we call $BoW(d)$ its corresponding **Bag of Words**.

$$P(d|c) \simeq P(BoW(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(BoW(d)|c)P(c)$$

- Can you think of a classification task and a specific document (=sentence) for which losing positional information would likely affect classification?

The Bag of Word assumption

- Documents are “featurized” as **the multiset of their words**. A “multiset” is unordered like a set, but keeps track of multiple occurrences.
- It’s like writing each individual word of your document on separate pieces of paper, and tossing them all into a bag. For any document d , we call $BoW(d)$ its corresponding **Bag of Words**.

$$P(d|c) \simeq P(BoW(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(BoW(d)|c)P(c)$$



- Can you think of a classification task and a specific document (=sentence) for which losing positional information would likely affect classification?

The Bag of Word assumption

- Documents are “featurized” as **the multiset of their words**. A “multiset” is unordered like a set, but keeps track of multiple occurrences.
- It’s like writing each individual word of your document on separate pieces of paper, and tossing them all into a bag. For any document d , we call $BoW(d)$ its corresponding **Bag of Words**.

$$P(d|c) \simeq P(BoW(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(BoW(d)|c)P(c)$$

- Can you think of a classification task and a specific document (=sentence) for which losing positional information would likely affect classification?

The independence assumption

$$P(d|c) \simeq P(\text{BoW}(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(\text{BoW}(d)|c)P(c)$$

- To further decompose $P(\text{BoW}(d)|c)$, we take that the words in our document are **produced independently from each other**.
- This means that knowing a word is in the bag does not inform us about how likely other words are to be in the bag.
- So our likelihood becomes a convenient product of conditional probabilities, one per feature, i.e. one per word in the bag.

$$\begin{aligned} P(d|c) \simeq P(\text{BoW}(d)|c) &= P(\{w_0, w_1, \dots, w_n\}|c) \\ &= P(w_0|c) \times P(w_1|c) \times \dots \times P(w_n|c) \\ &= \prod_{w \in \text{BoW}(d)} P(w|c) \end{aligned}$$

The independence assumption

$$P(d|c) \simeq P(\text{BoW}(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(\text{BoW}(d)|c)P(c)$$

- To further decompose $P(\text{BoW}(d)|c)$, we take that the words in our document are **produced independently from each other**.
- This means that knowing a word is in the bag does not inform us about how likely other words are to be in the bag.
- So our likelihood becomes a convenient product of conditional probabilities, one per feature, i.e. one per word in the bag.

$$\begin{aligned} P(d|c) \simeq P(\text{BoW}(d)|c) &= P(\{w_0, w_1, \dots, w_n\}|c) \\ &= P(w_0|c) \times P(w_1|c) \times \dots \times P(w_n|c) \\ &= \prod_{w \in \text{BoW}(d)} P(w|c) \end{aligned}$$

The independence assumption

$$P(d|c) \simeq P(\text{BoW}(d)|c)$$

$$\hat{c}(d) = \operatorname{argmax}_{c \in \{c_0, c_1\}} P(\text{BoW}(d)|c)P(c)$$

- To further decompose $P(\text{BoW}(d)|c)$, we take that the words in our document are **produced independently from each other**.
- This means that knowing a word is in the bag does not inform us about how likely other words are to be in the bag.
- So our likelihood becomes a convenient product of conditional probabilities, one per feature, i.e. one per word in the bag.

$$\begin{aligned} P(d|c) \simeq P(\text{BoW}(d)|c) &= P(\{w_0, w_1, \dots, w_n\}|c) \\ &= P(w_0|c) \times P(w_1|c) \times \dots \times P(w_n|c) \\ &= \prod_{w \in \text{BoW}(d)} P(w|c) \end{aligned}$$

Challenging the independence assumption

- Can you give arguments from syntax or semantics showing why the independence assumption is “naive”?

Challenging the independence assumption

- Can you give arguments from syntax or semantics showing why the independence assumption is “naive”?
 - Some words “need” to be in a question or under negation. They are called negative polarity items (NPIs).

(4) Have you heard **anything**_{NPI}? I haven't heard **anything**_{NPI}/I have heard **something**!

- Words with similar meanings tend to occur together.

(5) The **sailor steered** the **boat** through the **waves**, tightened the **sails**, and guided the **vessel** safely into the **harbour**. ©ChatGPT

- Verbs assign fixed roles to their arguments (Agent, Patient, Experiencer, Goal)...leading them to “prefer” certain nominals (agentive vs. passive, animate vs. inanimate...).
- English questions are often formed via *do*-support, along with a fronted *wh*-word. Also, subjects and verbs agree in number...

(6) **What did** you eat? I ate an apple.

Summary of the model

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- A word w occurring multiple times in $\text{BoW}(d)$ will be associated with multiple terms $P(w|c)$ in the above product.
- A document belongs to the class that maximizes the likelihood of its constitutive words ($P(w|c)$), weighted by the class' prior ($P(c)$).
- Next question: how to estimate $P(c)$ and $P(w|c)$ for any c and w ?

Summary of the model

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- A word w occurring multiple times in $\text{BoW}(d)$ will be associated with multiple terms $P(w|c)$ in the above product.
- A document belongs to the class that maximizes the likelihood of its constitutive words ($P(w|c)$), weighted by the class' prior ($P(c)$).
- Next question: how to estimate $P(c)$ and $P(w|c)$ for any c and w ?

Summary of the model

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- A word w occurring multiple times in $\text{BoW}(d)$ will be associated with multiple terms $P(w|c)$ in the above product.
- A document belongs to the class that maximizes the likelihood of its constitutive words ($P(w|c)$), weighted by the class' prior ($P(c)$).
- Next question: how to estimate $P(c)$ and $P(w|c)$ for any c and w ?

Estimating priors and likelihoods

A supervised learning task

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \times P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c) \\ &\quad (\text{with } \textit{BoW}(d) = \{w_1, \dots, w_n\}) \text{ (a multiset)} \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \textit{BoW}(d)} P(w|c)\end{aligned}$$

- To estimate our **priors** $P(c)$ and **likelihoods** $P(w|c)$ for each class c and word w , we'll use **statistics computed from a training dataset** $\mathcal{T}$.
- We'll assume the dataset is a set of labeled documents.
 $\mathcal{T} = \{(d_1, c(d_1)), (d_2, c(d_2)), \dots, (d_k, c(d_k))\}$.
- For instance, $\mathcal{T}$ can be a set of tweets, labeled as harmful or neutral.

A supervised learning task

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \times P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c) \\ &\quad (\text{with } \textit{BoW}(d) = \{w_1, \dots, w_n\}) \text{ (a multiset)} \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \textit{BoW}(d)} P(w|c)\end{aligned}$$

- To estimate our **priors** $P(c)$ and **likelihoods** $P(w|c)$ for each class c and word w , we'll use **statistics computed from a training dataset** $\mathcal{T}$.
- We'll assume the dataset is a set of labeled documents.
 $\mathcal{T} = \{(d_1, c(d_1)), (d_2, c(d_2)), \dots, (d_k, c(d_k))\}$.
- For instance, $\mathcal{T}$ can be a set of tweets, labeled as harmful or neutral.

A supervised learning task

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \times P(w_1|c) \times P(w_2|c) \times \dots \times P(w_n|c) \\ &\quad (\text{with } \textit{BoW}(d) = \{w_1, \dots, w_n\}) \text{ (a multiset)} \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \textit{BoW}(d)} P(w|c)\end{aligned}$$

- To estimate our **priors** $P(c)$ and **likelihoods** $P(w|c)$ for each class c and word w , we'll use **statistics computed from a training dataset** $\mathcal{T}$.
- We'll assume the dataset is a set of labeled documents.
 $\mathcal{T} = \{(d_1, c(d_1)), (d_2, c(d_2)), \dots, (d_k, c(d_k))\}$.
- For instance, $\mathcal{T}$ can be a set of tweets, labeled as harmful or neutral.

Priors

- We need to estimate our prior $P(c)$ for each class c , e.g. the probability for a document to be harmful/neutral **given no extra evidence**.
- To do this, we can look at which **proportion** of the documents of the dataset $\mathcal{T}$ are of class c_i , with $i = 0$ (neutral) or $i = 1$ (harmful). $|\cdot|$ is the set cardinality function—not the absolute value function.

$$\begin{aligned} P(c_i) &\simeq \frac{|\{(d, c(d)) \in \mathcal{T} \mid c(d) = c_i\}|}{|\mathcal{T}|} \\ &\simeq \frac{\text{number of docs of class } c_i \text{ in } \mathcal{T}}{\text{number of docs in } \mathcal{T}} \end{aligned}$$

- So, if $\mathcal{T}$ contains 87 harmful tweets and 1254 neutral ones:

$$\begin{aligned} P(c_0) &\simeq P(\text{neutral}) \\ &\simeq \frac{1254}{1254 + 87} \\ &\simeq 93.5\% \end{aligned}$$

$$\begin{aligned} P(c_1) &\simeq P(\text{harmful}) \\ &\simeq \frac{87}{1254 + 87} \\ &\simeq 6.5\% \end{aligned}$$

Priors

- We need to estimate our prior $P(c)$ for each class c , e.g. the probability for a document to be harmful/neutral **given no extra evidence**.
- To do this, we can look at which **proportion** of the documents of the dataset $\mathcal{T}$ are of class c_i , with $i = 0$ (neutral) or $i = 1$ (harmful). $|\cdot|$ is the set cardinality function—not the absolute value function.

$$\begin{aligned} P(c_i) &\simeq \frac{|\{(d, c(d)) \in \mathcal{T} \mid c(d) = c_i\}|}{|\mathcal{T}|} \\ &\simeq \frac{\text{number of docs of class } c_i \text{ in } \mathcal{T}}{\text{number of docs in } \mathcal{T}} \end{aligned}$$

- So, if $\mathcal{T}$ contains 87 harmful tweets and 1254 neutral ones:

$$\begin{aligned} P(c_0) &\simeq P(\text{neutral}) \\ &\simeq \frac{1254}{1254 + 87} \\ &\simeq 93.5\% \end{aligned}$$

$$\begin{aligned} P(c_1) &\simeq P(\text{harmful}) \\ &\simeq \frac{87}{1254 + 87} \\ &\simeq 6.5\% \end{aligned}$$

Priors

- We need to estimate our prior $P(c)$ for each class c , e.g. the probability for a document to be harmful/neutral **given no extra evidence**.
- To do this, we can look at which **proportion** of the documents of the dataset $\mathcal{T}$ are of class c_i , with $i = 0$ (neutral) or $i = 1$ (harmful). $|\cdot|$ is the set cardinality function—not the absolute value function.

$$\begin{aligned} P(c_i) &\simeq \frac{|\{(d, c(d)) \in \mathcal{T} \mid c(d) = c_i\}|}{|\mathcal{T}|} \\ &\simeq \frac{\text{number of docs of class } c_i \text{ in } \mathcal{T}}{\text{number of docs in } \mathcal{T}} \end{aligned}$$

- So, if $\mathcal{T}$ contains 87 harmful tweets and 1254 neutral ones:

$$\begin{aligned} P(c_0) &\simeq P(\text{neutral}) \\ &\simeq \frac{1254}{1254 + 87} \\ &\simeq 93.5\% \end{aligned}$$

$$\begin{aligned} P(c_1) &\simeq P(\text{harmful}) \\ &\simeq \frac{87}{1254 + 87} \\ &\simeq 6.5\% \end{aligned}$$

Feature likelihoods step 1: mega-docs

- We need to estimate how likely any feature (=word) is, **given a class**: $P(w|c)$.
- For example, we need to estimate how likely we are to see the word *nice* in a harmful vs. a neutral document.
- For any class c , we define d_c as the “mega-document” resulting from the concatenation of all documents of class c in $\mathcal{T}$.
- d_c is a multiset: it may contain several copies of the same word.

$$\begin{aligned}d_{c_i} &= \bigsqcup \{ \text{BoW}(d) \mid (d, c(d)) \in \mathcal{T} \text{ and } c(d) = c_i \} \\ &\simeq \text{the big compilation of all datapoints of class } c_i\end{aligned}$$

- Intuition: a word like *nice* will be **more frequent** ($\simeq$ more likely) in the mega-doc d_{c_0} associated with the **neutral** class, as opposed to the mega-doc d_{c_1} associated with the harmful class!

Feature likelihoods step 1: mega-docs

- We need to estimate how likely any feature (=word) is, **given a class**: $P(w|c)$.
- For example, we need to estimate how likely we are to see the word *nice* in a harmful vs. a neutral document.
- For any class c , we define d_c as the “mega-document” resulting from the concatenation of all documents of class c in $\mathcal{T}$.
- d_c is a multiset: it may contain several copies of the same word.

$$\begin{aligned} d_{c_i} &= \bigsqcup \{ \text{BoW}(d) \mid (d, c(d)) \in \mathcal{T} \text{ and } c(d) = c_i \} \\ &\simeq \text{the big compilation of all datapoints of class } c_i \end{aligned}$$

- Intuition: a word like *nice* will be **more frequent** ($\simeq$ more likely) in the mega-doc d_{c_0} associated with the **neutral** class, as opposed to the mega-doc d_{c_1} associated with the harmful class!

Feature likelihoods step 1: mega-docs

- We need to estimate how likely any feature (=word) is, **given a class**: $P(w|c)$.
- For example, we need to estimate how likely we are to see the word *nice* in a harmful vs. a neutral document.
- For any class c , we define d_c as the “mega-document” resulting from the concatenation of all documents of class c in $\mathcal{T}$.
- d_c is a multiset: it may contain several copies of the same word.

$$\begin{aligned} d_{c_i} &= \bigsqcup \{ \text{BoW}(d) \mid (d, c(d)) \in \mathcal{T} \text{ and } c(d) = c_i \} \\ &\simeq \text{the big compilation of all datapoints of class } c_i \end{aligned}$$

- Intuition: a word like *nice* will be more frequent ($\simeq$ more likely) in the mega-doc d_{c_0} associated with the neutral class, as opposed to the mega-doc d_{c_1} associated with the harmful class!

Feature likelihoods step 1: mega-docs

- We need to estimate how likely any feature (=word) is, **given a class**: $P(w|c)$.
- For example, we need to estimate how likely we are to see the word *nice* in a harmful vs. a neutral document.
- For any class c , we define d_c as the “mega-document” resulting from the concatenation of all documents of class c in $\mathcal{T}$.
- d_c is a multiset: it may contain several copies of the same word.

$$\begin{aligned} d_{c_i} &= \bigsqcup \{ \text{BoW}(d) \mid (d, c(d)) \in \mathcal{T} \text{ and } c(d) = c_i \} \\ &\simeq \text{the big compilation of all datapoints of class } c_i \end{aligned}$$

- Intuition: a word like *nice* will be **more frequent** ($\simeq$ **more likely**) in the mega-doc d_{c_0} associated with the **neutral** class, as opposed to the mega-doc d_{c_1} associated with the harmful class!

Feature likelihoods step 2: frequencies in mega-docs

- We can then estimate $P(w|c)$ using the frequency of w , within the mega-document d_c .

$$\begin{aligned}P(w|c_i) &\simeq \frac{C(w, d_{c_i})}{|d_{c_i}|} && \text{with } C \text{ the count function} \\&\simeq \frac{\text{Number of occurrences of } w \text{ in the mega-doc } d_{c_i}}{\text{Total size of } d_{c_i}}\end{aligned}$$

- Suppose that together, all the harmful tweets in $\mathcal{T}$ add up to 2610 words, with 3 occurrences of *nice*. All the neutral tweets add up to 37625 words, with 455 occurrences of *nice*.

$$\begin{aligned}P(\textit{nice}|c_0) &\simeq P(\textit{nice}|\textit{neutral}) && P(\textit{nice}|c_1) \simeq P(\textit{nice}|\textit{harmful}) \\&\simeq \frac{455}{37625} && \simeq \frac{3}{2610} \\&\simeq 12.1\text{‰} && \simeq 1.15\text{‰}\end{aligned}$$

Feature likelihoods step 2: frequencies in mega-docs

- We can then estimate $P(w|c)$ using the frequency of w , within the mega-document d_c .

$$\begin{aligned}P(w|c_i) &\simeq \frac{C(w, d_{c_i})}{|d_{c_i}|} && \text{with } C \text{ the count function} \\&\simeq \frac{\text{Number of occurrences of } w \text{ in the mega-doc } d_{c_i}}{\text{Total size of } d_{c_i}}\end{aligned}$$

- Suppose that together, all the harmful tweets in $\mathcal{T}$ add up to 2610 words, with 3 occurrences of *nice*. All the neutral tweets add up to 37625 words, with 455 occurrences of *nice*.

$$\begin{aligned}P(\textit{nice}|c_0) &\simeq P(\textit{nice}|\textit{neutral}) && P(\textit{nice}|c_1) \simeq P(\textit{nice}|\textit{harmful}) \\&\simeq \frac{455}{37625} && \simeq \frac{3}{2610} \\&\simeq 12.1\text{‰} && \simeq 1.15\text{‰}\end{aligned}$$

Summary of the naive Bayes approach

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- The class we assign to a document is the “most likely”, **given some relevant features of the document**—typically the words it contains.
- What counts as “most likely” depends on **how likely each feature of the document is**, given the class ($P(w|c)$), but also on **how probable the class is overall** (the prior $P(c)$).
- Both $P(c)$ and $P(w|c)$ can be estimated by extracting document/word **frequencies** from a training dataset.

Summary of the naive Bayes approach

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- The class we assign to a document is the “most likely”, **given some relevant features of the document**—typically the words it contains.
- What counts as “most likely” depends on **how likely each feature of the document is**, given the class ($P(w|c)$), but also on **how probable the class is overall** (the prior $P(c)$).
- Both $P(c)$ and $P(w|c)$ can be estimated by extracting document/word **frequencies** from a training dataset.

Summary of the naive Bayes approach

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- The class we assign to a document is the “most likely”, **given some relevant features of the document**—typically the words it contains.
- What counts as “most likely” depends on **how likely each feature of the document is**, given the class ($P(w|c)$), but also on **how probable the class is** overall (the prior $P(c)$).
- Both $P(c)$ and $P(w|c)$ can be estimated by extracting document/word **frequencies** from a training dataset.

Summary of the naive Bayes approach

$$\begin{aligned}\hat{c}(d) &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c|d) \\ &= \operatorname{argmax}_{c \in \{c_0, c_1\}} P(c) \prod_{w \in \text{BoW}(d)} P(w|c)\end{aligned}$$

- The class we assign to a document is the “most likely”, **given some relevant features of the document**—typically the words it contains.
- What counts as “most likely” depends on **how likely each feature of the document is**, given the class ($P(w|c)$), but also on **how probable the class is** overall (the prior $P(c)$).
- Both $P(c)$ and $P(w|c)$ can be estimated by extracting document/word **frequencies** from a training dataset.

Evaluating a Naive Bayes model

Defining a test set

- To evaluate a Naive Bayes model, we'll need fresh, annotated data, in the form of a **test set** $\mathcal{T}'$.
- This set will have **same structure** as the training set: a set of document-class pairs.
- $\mathcal{T}'$ must be similar-enough to the training set to enable **meaningful comparisons** between competing models/sets of relevant features.
- But $\mathcal{T}'$ must also contain fresh unseen data, in order to estimate how well the model **generalized** beyond the training data.
- The metrics we'll see in this section are **extremely general**, and apply beyond Naive Bayes! In fact they extend to any binary classifier!

Defining a test set

- To evaluate a Naive Bayes model, we'll need fresh, annotated data, in the form of a **test set** $\mathcal{T}'$.
- This set will have **same structure** as the training set: a set of document-class pairs.
- $\mathcal{T}'$ must be similar-enough to the training set to enable **meaningful comparisons** between competing models/sets of relevant features.
- But $\mathcal{T}'$ must also contain fresh unseen data, in order to estimate how well the model **generalized** beyond the training data.
- The metrics we'll see in this section are **extremely general**, and apply beyond Naive Bayes! In fact they extend to any binary classifier!

Defining a test set

- To evaluate a Naive Bayes model, we'll need fresh, annotated data, in the form of a **test set** $\mathcal{T}'$.
- This set will have **same structure** as the training set: a set of document-class pairs.
- $\mathcal{T}'$ must be similar-enough to the training set to enable **meaningful comparisons** between competing models/sets of relevant features.
- But $\mathcal{T}'$ must also contain fresh unseen data, in order to estimate how well the model **generalized** beyond the training data.
- The metrics we'll see in this section are **extremely general**, and apply beyond Naive Bayes! In fact they extend to any binary classifier!

Defining a test set

- To evaluate a Naive Bayes model, we'll need fresh, annotated data, in the form of a **test set** $\mathcal{T}'$.
- This set will have **same structure** as the training set: a set of document-class pairs.
- $\mathcal{T}'$ must be similar-enough to the training set to enable **meaningful comparisons** between competing models/sets of relevant features.
- But $\mathcal{T}'$ must also contain fresh unseen data, in order to estimate how well the model **generalized** beyond the training data.
- The metrics we'll see in this section are extremely general, and apply beyond Naive Bayes! In fact they extend to any binary classifier!

Defining a test set

- To evaluate a Naive Bayes model, we'll need fresh, annotated data, in the form of a **test set** $\mathcal{T}'$.
- This set will have **same structure** as the training set: a set of document-class pairs.
- $\mathcal{T}'$ must be similar-enough to the training set to enable **meaningful comparisons** between competing models/sets of relevant features.
- But $\mathcal{T}'$ must also contain fresh unseen data, in order to estimate how well the model **generalized** beyond the training data.
- **The metrics we'll see in this section are extremely general, and apply beyond Naive Bayes! In fact they extend to any binary classifier!**

An intuitive measure of success: accuracy

- We have $\mathcal{T}' = \{(d'_1, c(d'_1)), (d'_2, c(d'_2)), \dots, (d'_k, c(d'_k))\}$.
- Intuitive idea: for each pair $((d'_i, c(d'_i)))$, predict $\hat{c}(d'_i)$ using the Naive Bayes model, and compare $\hat{c}(d'_i)$ to the ground truth $c(d'_i)$:
 - If they match yay! We increment our success score by 1;
 - If they don't match shoot! We move on to the next pair.
- We can then compute our probability of success: our success score divided by the number of pairs scanned (k).
- More formally, the **accuracy** of a Naive Bayes model (i.e. a classification function $\hat{c}$) on a test set $\mathcal{T}'$ is:

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{|\{(d', c') \in \mathcal{T}' \mid c' = \hat{c}(d')\}|}{|\mathcal{T}'|} \\ &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}}\end{aligned}$$

An intuitive measure of success: accuracy

- We have $\mathcal{T}' = \{(d'_1, c(d'_1)), (d'_2, c(d'_2)), \dots, (d'_k, c(d'_k))\}$.
- Intuitive idea: for each pair $((d'_i, c(d'_i)))$, predict $\hat{c}(d'_i)$ using the Naive Bayes model, and compare $\hat{c}(d'_i)$ to the ground truth $c(d'_i)$:
 - If they match yay! We increment our success score by 1;
 - If they don't match shoot! We move on to the next pair.
- We can then compute our probability of success: our success score divided by the number of pairs scanned (k).
- More formally, the **accuracy** of a Naive Bayes model (i.e. a classification function $\hat{c}$) on a test set $\mathcal{T}'$ is:

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{|\{(d', c') \in \mathcal{T}' \mid c' = \hat{c}(d')\}|}{|\mathcal{T}'|} \\ &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}}\end{aligned}$$

An intuitive measure of success: accuracy

- We have $\mathcal{T}' = \{(d'_1, c(d'_1)), (d'_2, c(d'_2)), \dots, (d'_k, c(d'_k))\}$.
- Intuitive idea: for each pair $((d'_i, c(d'_i)))$, predict $\hat{c}(d'_i)$ using the Naive Bayes model, and compare $\hat{c}(d'_i)$ to the ground truth $c(d'_i)$:
 - If they match yay! We increment our success score by 1;
 - If they don't match shoot! We move on to the next pair.
- We can then compute our probability of success: our success score divided by the number of pairs scanned (k).
- More formally, the **accuracy** of a Naive Bayes model (i.e. a classification function $\hat{c}$) on a test set $\mathcal{T}'$ is:

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{|\{(d', c') \in \mathcal{T}' \mid c' = \hat{c}(d')\}|}{|\mathcal{T}'|} \\ &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}}\end{aligned}$$

An intuitive measure of success: accuracy

- We have $\mathcal{T}' = \{(d'_1, c(d'_1)), (d'_2, c(d'_2)), \dots, (d'_k, c(d'_k))\}$.
- Intuitive idea: for each pair $((d'_i, c(d'_i)))$, predict $\hat{c}(d'_i)$ using the Naive Bayes model, and compare $\hat{c}(d'_i)$ to the ground truth $c(d'_i)$:
 - If they match yay! We increment our success score by 1;
 - If they don't match shoot! We move on to the next pair.
- We can then compute our probability of success: our success score divided by the number of pairs scanned (k).
- More formally, the accuracy of a Naive Bayes model (i.e. a classification function $\hat{c}$) on a test set $\mathcal{T}'$ is:

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{|\{(d', c') \in \mathcal{T}' \mid c' = \hat{c}(d')\}|}{|\mathcal{T}'|} \\ &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}}\end{aligned}$$

An intuitive measure of success: accuracy

- We have $\mathcal{T}' = \{(d'_1, c(d'_1)), (d'_2, c(d'_2)), \dots, (d'_k, c(d'_k))\}$.
- Intuitive idea: for each pair $((d'_i, c(d'_i)))$, predict $\hat{c}(d'_i)$ using the Naive Bayes model, and compare $\hat{c}(d'_i)$ to the ground truth $c(d'_i)$:
 - If they match yay! We increment our success score by 1;
 - If they don't match shoot! We move on to the next pair.
- We can then compute our probability of success: our success score divided by the number of pairs scanned (k).
- More formally, the **accuracy** of a Naive Bayes model (i.e. a classification function $\hat{c}$) on a test set $\mathcal{T}'$ is:

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{|\{(d', c') \in \mathcal{T}' \mid c' = \hat{c}(d')\}|}{|\mathcal{T}'|} \\ &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}}\end{aligned}$$

Shortcomings of accuracy: the stupid classifier

- Suppose our two classes are **very imbalanced**, as is often the case: the more critical/problematic class is much rarer.

Shortcomings of accuracy: the stupid classifier

- Suppose our two classes are **very imbalanced**, as is often the case: the more critical/problematic class is much rarer.
- For instance, assume $\mathcal{T}'$ contains 90% of neutral tweets, and 10% of harmful tweets.

Shortcomings of accuracy: the stupid classifier

- Suppose our two classes are **very imbalanced**, as is often the case: the more critical/problematic class is much rarer.
- For instance, assume $\mathcal{T}'$ contains 90% of neutral tweets, and 10% of harmful tweets.
- Take a very **stupid classifier**, predicting a tweet to be neutral 100% of the time. What will be its accuracy?

Shortcomings of accuracy: the stupid classifier

- Suppose our two classes are **very imbalanced**, as is often the case: the more critical/problematic class is much rarer.
- For instance, assume $\mathcal{T}'$ contains 90% of neutral tweets, and 10% of harmful tweets.
- Take a very **stupid classifier**, predicting a tweet to be neutral 100% of the time. What will be its accuracy?

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}_{\text{stupid}}) &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}} \\ &= \frac{\text{Number of neutral datapoints}}{\text{Number of datapoints}} \\ &= 90\%\end{aligned}$$

Shortcomings of accuracy: the stupid classifier

- Suppose our two classes are **very imbalanced**, as is often the case: the more critical/problematic class is much rarer.
- For instance, assume $\mathcal{T}'$ contains 90% of neutral tweets, and 10% of harmful tweets.
- Take a very **stupid classifier**, predicting a tweet to be neutral 100% of the time. What will be its accuracy?

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{\mathbf{c}}_{\text{stupid}}) &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}} \\ &= \frac{\text{Number of neutral datapoints}}{\text{Number of datapoints}} \\ &= 90\%\end{aligned}$$

- Pretty high! But what's the point if harmful tweets are never identified?

Shortcomings of accuracy: the zealous classifier

- Now consider $\hat{c}$, a classifier which at least tries to identify harmful tweets—causing it to mispredict some neutral ones.

Shortcomings of accuracy: the zealous classifier

- Now consider $\hat{c}$, a classifier which at least tries to identify harmful tweets—causing it to mispredict some neutral ones.
- Let's assume 5/10 harmful tweets, and 80/90 neutral tweets from $\mathcal{T}'$ are correctly classified by $\hat{c}$. What will be its accuracy?

Shortcomings of accuracy: the zealous classifier

- Now consider $\hat{c}$, a classifier which at least tries to identify harmful tweets—causing it to mispredict some neutral ones.
- Let's assume 5/10 harmful tweets, and 80/90 neutral tweets from $\mathcal{T}'$ are correctly classified by $\hat{c}$. What will be its accuracy?

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}} \\ &= \frac{5 + 80}{100} \\ &= 85\%\end{aligned}$$

Shortcomings of accuracy: the zealous classifier

- Now consider $\hat{c}$, a classifier which at least tries to identify harmful tweets—causing it to mispredict some neutral ones.
- Let's assume 5/10 harmful tweets, and 80/90 neutral tweets from $\mathcal{T}'$ are correctly classified by $\hat{c}$. What will be its accuracy?

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}} \\ &= \frac{5 + 80}{100} \\ &= 85\%\end{aligned}$$

- That's lower than the stupid classifier :(

Shortcomings of accuracy: the zealous classifier

- Now consider $\hat{c}$, a classifier which at least tries to identify harmful tweets—causing it to mispredict some neutral ones.
- Let's assume 5/10 harmful tweets, and 80/90 neutral tweets from $\mathcal{T}'$ are correctly classified by $\hat{c}$. What will be its accuracy?

$$\begin{aligned}\text{ACCURACY}(\mathcal{T}', \hat{c}) &= \frac{\text{Number of datapoints whose class was correctly predicted}}{\text{Number of datapoints}} \\ &= \frac{5 + 80}{100} \\ &= 85\%\end{aligned}$$

- That's lower than the stupid classifier :((
- Intuition: **had we focused on the harmful tweets**, we'd have noticed that the stupid classifier was doing a poor job (none of them were identified), and the zealous classifier a better job (5 were identified).

Recall

- We want a measure that tracks some proportion of correctly predicted harmful tweets.
- One idea is to look at the proportion of correctly predicted harmful tweets, over all *actually* harmful tweets.
- This is known as **recall**.
- The stupid classifier predicted 0 tweets to be harmful, so its recall is 0!
- The zealous classifier correctly predicted 5 tweets to be harmful, out of the 10 actually harmful tweets, so its recall is 50%! Better!

Recall

- We want a measure that tracks some proportion of correctly predicted harmful tweets.
- One idea is to look at the **proportion of correctly predicted harmful tweets, over all *actually* harmful tweets.**
- This is known as **recall**.
- The stupid classifier predicted 0 tweets to be harmful, so its recall is 0!
- The zealous classifier correctly predicted 5 tweets to be harmful, out of the 10 actually harmful tweets, so its recall is 50%! Better!

Recall

- We want a measure that tracks some proportion of correctly predicted harmful tweets.
- One idea is to look at the **proportion of correctly predicted harmful tweets, over all *actually* harmful tweets.**
- This is known as **recall**.
- The stupid classifier predicted 0 tweets to be harmful, so its recall is 0!
- The zealous classifier correctly predicted 5 tweets to be harmful, out of the 10 actually harmful tweets, so its recall is 50%! Better!

Recall

- We want a measure that tracks some proportion of correctly predicted harmful tweets.
- One idea is to look at the **proportion of correctly predicted harmful tweets, over all *actually* harmful tweets.**
- This is known as **recall**.
- The stupid classifier predicted 0 tweets to be harmful, so its recall is 0!
- The zealous classifier correctly predicted 5 tweets to be harmful, out of the 10 actually harmful tweets, so its recall is 50%! Better!

Precision

- Another way to track the proportion of correctly predicted harmful tweets, is to look at the **proportion of correctly predicted harmful tweets, over all tweets that were *predicted* to be harmful—correctly or not.**
- This is known as **precision**.
- The stupid classifier predicted 0 tweets to be harmful, so its precision is undefined (null numerator and denominator).
- The zealous classifier correctly predicted 5 tweets to be harmful, and incorrectly predicted 10 tweets to be harmful, so its precision is $5/(5 + 10) = 33\%$! Better!

Precision

- Another way to track the proportion of correctly predicted harmful tweets, is to look at the **proportion of correctly predicted harmful tweets, over all tweets that were *predicted* to be harmful—correctly or not.**
- This is known as **precision**.
- The stupid classifier predicted 0 tweets to be harmful, so its precision is undefined (null numerator and denominator).
- The zealous classifier correctly predicted 5 tweets to be harmful, and incorrectly predicted 10 tweets to be harmful, so its precision is $5/(5 + 10) = 33\%$! Better!

Precision

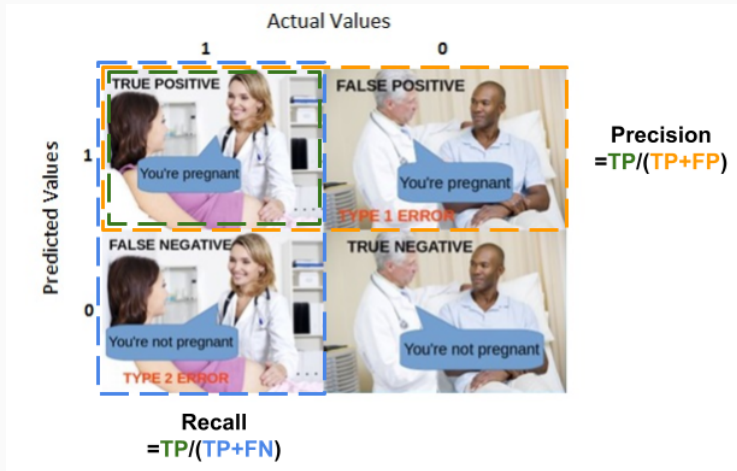
- Another way to track the proportion of correctly predicted harmful tweets, is to look at the **proportion of correctly predicted harmful tweets, over all tweets that were *predicted* to be harmful—correctly or not.**
- This is known as **precision**.
- The stupid classifier predicted 0 tweets to be harmful, so its precision is undefined (null numerator and denominator).
- The zealous classifier correctly predicted 5 tweets to be harmful, and incorrectly predicted 10 tweets to be harmful, so its precision is $5/(5 + 10) = 33\%$! Better!

Precision

- Another way to track the proportion of correctly predicted harmful tweets, is to look at the **proportion of correctly predicted harmful tweets, over all tweets that were *predicted* to be harmful—correctly or not.**
- This is known as **precision**.
- The stupid classifier predicted 0 tweets to be harmful, so its precision is undefined (null numerator and denominator).
- The zealous classifier correctly predicted 5 tweets to be harmful, and incorrectly predicted 10 tweets to be harmful, so its precision is $5/(5 + 10) = 33\%$! Better!

Precision and recall summary

- Imo the definitions of precision and recall are hard to remember. Memorable examples/graphical representations can help.



- Lastly, there's a way to combine precision and recall into one score balancing the two. This is known as the F-score.

$$F = \frac{2 \times \text{PRECISION} \times \text{RECALL}}{\text{PRECISION} + \text{RECALL}}$$

https://colab.research.google.com/drive/1TprbFZxk-LLtzh22MoCGSNb4lNPZL_AOH?usp=sharing